第 3 章注意力机制零基础问答讲义

基于 `ch03/01_main-chapter-code/ch03.ipynb` 重组

2026-04-20

目录

1 这份讲义到底在做什么 **2**

2 新的讲解原则 **2**

3 整章主线和新手最该问的问题 **2**

4 问题一：为什么会需要 attention **3**

5 问题二：极简 self-attention 到底在算什么 **3**

5.1 先说这一节的目标 3

5.2 新手会先问什么 4

5.3 逐个回答 4

5.4 为什么点积和 softmax 很自然 5

5.5 这一步和真实模型还差什么 5

6 问题三：为什么真实模型要先变成 Q、K、V **6**

7 问题四：为什么语言模型必须加 causal mask **7**

8 问题五：为什么要从单头变成 multi-head **8**

9 把 notebook 的代码主线整理成一张表 **10**

10 这一章和真实 GPT 之间还差什么 **10**

11 最后真正该记住什么 **10**

1 这份讲义到底在做什么

这一部分在解决什么

原 notebook 的材料是按“图 + 解释 + 代码”铺开的。对于零基础读者，这样读很容易出现两个问题：第一，不知道自己现在最该问什么；第二，公式、代码、图之间对不上。这份讲义把它重组为一条问题链：先问为什么需要 attention，再问 self-attention 在算什么，再问为什么要有 Q、K、V、causal mask 和 multi-head，最后再回到代码。

本讲义不追求逐段翻译 notebook。它只做一件更有用的事：把第 3 章整理成“零基础可以顺着提问学下去”的版本。

2 新的讲解原则

后面每个知识点都按四层推进：

1. **背景问题**：它在解决什么现实困难。
2. **朴素直觉**：先不写公式，它到底在做什么。
3. **数学表达**：公式里每个量分别是什么。
4. **代码对应**：notebook 里哪一段代码在做这件事。

这意味着：

- 不等你逐图提问；
- 不默认你已经懂 embedding、softmax、query、key、value；
- 不把玩具数字和语义分开讲；
- 不把极简版和真实模型混在一起。

3 整章主线和新手最该问的问题

这一章的主线其实很短：

长序列难处理 → 先学极简 self-attention → 引入 Q/K/V 变成可训练模块 → 加 causal mask 变成语言模型可用版本 → 扩展成 multi-head attention

如果你现在是零基础，这一章最该问的不是“某张图是什么意思”，而是下面这五个问题：

1. 为什么处理长序列会逼出 attention?
2. self-attention 说“一个词去看别的词”，到底在算什么?
3. 为什么真实模型不能直接用原始输入，而要先变成 Q、K、V?
4. 为什么 GPT 这类语言模型必须加 causal mask?
5. 为什么还要把单头 attention 扩展成 multi-head?

4 问题一：为什么会需要 attention

这一部分在解决什么

在 attention 出现之前，机器翻译常用的是 RNN encoder-decoder。它的核心做法是：编码器先把整句输入压缩成一个 hidden state，再让解码器根据这个压缩表示逐词生成输出。问题是，句子一长，这个固定长度的压缩向量就容易丢信息。

先抓直觉

可以把旧方法想成“先把整句话背成一张小抄，再开始翻译”。句子短时，小抄还够用；句子长时，细节会被挤掉。attention 的思路更像“翻译当前词时，允许你回头看原句里所有位置，但不是平均看，而是按相关性有选择地看”。

数学表达

旧的 encoder-decoder 常把整句输入压成一个上下文向量 c：

$$c = h_n.$$

这里的 h_n 是编码器读完整个输入序列后得到的最后一个 hidden state。解码器再去建模：

$$P(y_1, \ldots, y_m \mid x_1, \ldots, x_n) = \prod_{t=1}^{m} P(y_t \mid y_{<t}, c).$$

这套写法的问题不在公式形式，而在于：**所有输入信息都被迫塞进同一个 c 里**。attention 的改动是，把“单个 c”换成“每一步动态算一个 c_t”。

代码对应

notebook 这一部分主要是概念铺垫，对应前面的图示和文字单元，没有代码：

- `Cell 8--16`：解释长序列翻译为什么需要跨位置依赖；
- `Cell 15`：把话题从一般 attention 过渡到 Transformer 里的 self-attention。

这一节真正该记住什么

attention 的动机不是“公式更复杂”，而是“不要再把整句死塞进一个固定向量里，而是在需要时回头看相关位置”。

5 问题二：极简 self-attention 到底在算什么

5.1 先说这一节的目标

这一部分在解决什么

这一步先不引入可训练参数。目标只有一个：看懂 self-attention 的最小计算链。notebook 用一句很短的话 `Your journey starts with one step` 做演示，并把每个词先表示成一个 3 维向量。

位置	词	3 维向量
$x^{(1)}$	Your	$[0.43, 0.15, 0.89]$
$x^{(2)}$	journey	$[0.55, 0.87, 0.66]$
$x^{(3)}$	starts	$[0.57, 0.85, 0.64]$
$x^{(4)}$	with	$[0.22, 0.58, 0.33]$
$x^{(5)}$	one	$[0.77, 0.25, 0.10]$
$x^{(6)}$	step	$[0.05, 0.80, 0.55]$

这里的 $x^{(i)}$ 表示“第 i 个词的输入向量”，不是幂。

5.2 新手会先问什么

1. “一个词看别的词”里的“看”，到底是怎么算的？
2. 为什么先用点积打分？
3. 为什么打完分还要 softmax？
4. 最后的上下文向量又是什么？

5.3 逐个回答

先抓直觉

先只盯第二个词 $x^{(2)}$，也就是 `journey`。我们想更新它的表示时，不只想保留它自己，还想把全句里和它有关的信息揉进来。最自然的做法是：

1. 先让它和全句每个词都算一个相关性分数；
2. 再把这些分数变成“权重”；
3. 最后用这些权重对所有词向量做加权平均。

数学表达

在极简版里，直接拿第二个词自己的向量当 query：

$$q^{(2)} = x^{(2)}.$$

然后算它和每个输入向量的点积，得到未归一化分数：

$$\omega_{2j} = x^{(j)} \cdot q^{(2)}.$$

这里的 ω_{2j} 表示“当我们更新第 2 个词时，第 j 个词和它有多相关”。notebook 算出来的 6 个分数是

$$[0.9544,\ 1.4950,\ 1.4754,\ 0.8434,\ 0.7070,\ 1.0865].$$

再用 softmax 把分数变成和为 1 的权重：

$$\alpha_{2j} = \frac{e^{\omega_{2j}}}{\sum_{k=1}^{6} e^{\omega_{2k}}}.$$

这里的 α_{2j} 才是真正的 attention weight。notebook 得到的是

$$[0.1385,\ 0.2379,\ 0.2333,\ 0.1240,\ 0.1082,\ 0.1581].$$

最后把所有输入向量按这些权重做加权求和：

$$z^{(2)} = \sum_{j=1}^{6} \alpha_{2j} x^{(j)}.$$

这里的 $z^{(2)}$ 表示“第 2 个词的新上下文表示”。notebook 的结果是

$$z^{(2)} = [0.4419,\ 0.6515,\ 0.5683].$$

5.4 为什么点积和 softmax 很自然

- 点积大，表示两个向量更对齐，也就更相关。
- softmax 把任意实数分数变成非负、且总和为 1 的权重。
- 加权求和让当前词既保留自己，也吸收别的相关词的信息。

代码对应

这部分可以直接对照以下 code cell：

- `Cell 24`：定义 6 个词的 toy embedding；
- `Cell 28`：算第二个词对全句的原始分数；
- `Cell 37`：用 `torch.softmax` 变成注意力权重；
- `Cell 40`：算 $z^{(2)}$；
- `Cell 49, 51, 55`：把单个词推广到整句话所有词。

```
query = inputs[1]
attn_scores_2 = torch.empty(inputs.shape[0])
for i, x_i in enumerate(inputs):
    attn_scores_2[i] = torch.dot(x_i, query)

attn_weights_2 = torch.softmax(attn_scores_2, dim=0)
context_vec_2 = torch.zeros(query.shape)
for i, x_i in enumerate(inputs):
    context_vec_2 += attn_weights_2[i] * x_i
```

5.5 这一步和真实模型还差什么

这一节故意省略了三件事：

1. 没有可训练参数；
2. 没有区分 query、key、value；

3. 没有因果遮罩，所以还不能直接拿来做 GPT 式的下一个词预测。

这一节真正该记住什么

极简 self-attention 的骨架只有三步：**相关性打分 → softmax 归一化 → 加权求和**。

6 问题三：为什么真实模型要先变成 Q、K、V

这一部分在解决什么

如果直接拿原始输入向量互相做点积，模型几乎没有学习空间。真实模型需要自己学会“什么叫相关”“该取什么信息”，这就是 Q、K、V 的来历。

先抓直觉

可以把它想成三个不同视角：

- Query：当前词现在想找什么；
- Key：每个词身上提供了什么“可被匹配”的线索；
- Value：每个词真正携带、最后会被取走的信息。

如果没有这三套投影，模型只能在原始 embedding 空间里硬算相关性，表达能力会弱很多。

数学表达

对每个输入词 $x^{(i)}$，先乘三组可训练矩阵：

$$q^{(i)} = x^{(i)} W_Q, \qquad k^{(i)} = x^{(i)} W_K, \qquad v^{(i)} = x^{(i)} W_V.$$

这里：

- W_Q 学“当前词该怎么提问”；
- W_K 学“每个词该怎么被匹配”；
- W_V 学“每个词该输出什么内容”。

然后第 i 个词对第 j 个词的分数变成

$$s_{ij} = q^{(i)} \cdot k^{(j)}.$$

再做缩放和 softmax：

$$\alpha_{ij} = \mathrm{softmax}\left(\frac{s_{ij}}{\sqrt{d_k}}\right).$$

这里的 d_k 是 key 向量的维度。除以 $\sqrt{d_k}$ 是为了避免维度变大时点积数值过大，导致 softmax 过于尖锐、训练不稳定。

最后上下文向量变成

$$z^{(i)} = \sum_j \alpha_{ij} v^{(j)}.$$

注意：**加权求和时用的是 value，不是原始输入**。

notebook 用 $d_{\text{in}} = 3$、$d_{\text{out}} = 2$ 做了一个很小的演示。对第二个词，算出的关键数字是：

- $q^{(2)} = [0.4306,\ 1.4551]$；
- 原始分数 s_{2j} 为 $[1.2705,\ 1.8524,\ 1.8111,\ 1.0795,\ 0.5577,\ 1.5440]$；
- 缩放 softmax 后的权重为 $[0.1500,\ 0.2264,\ 0.2199,\ 0.1311,\ 0.0906,\ 0.1820]$；
- 最终 $z^{(2)} = [0.3061,\ 0.8210]$。

代码对应

notebook 这一段对应：

- `Cell 66--83`：手工一步步计算 Q、K、V、分数、权重、上下文；
- `Cell 86`：`SelfAttention_v1`，手写参数矩阵；
- `Cell 89`：`SelfAttention_v2`，用 `nn.Linear` 改写，更接近真实工程写法。

```
keys = self.W_key(x)
queries = self.W_query(x)
values = self.W_value(x)

attn_scores = queries @ keys.T
attn_weights = torch.softmax(
    attn_scores / keys.shape[-1]**0.5, dim=-1
)
context_vec = attn_weights @ values
```

这一节真正该记住什么

Q、K、V 不是为了把公式写复杂，而是为了让模型能学会“怎么提问、怎么匹配、怎么取值”。

7 问题四：为什么语言模型必须加 causal mask

这一部分在解决什么

如果我们在做“预测下一个词”，模型在计算当前位置时就绝不能偷看未来词。否则训练时它相当于提前看答案，推理时却看不到，训练和部署会不一致。

先抓直觉

对第 t 个位置来说，它最多只能看第 1 到第 t 个词，不能看第 $t+1$ 个及之后的词。causal mask 的作用就是把未来位置的注意力分数强行封死。

数学表达

设 M_{ij} 是遮罩矩阵：

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

那么 masked attention 写成

$$\alpha_{ij} = \mathrm{softmax}\left(\frac{s_{ij} + M_{ij}}{\sqrt{d_k}}\right).$$

这里把未来位置设成 $-\infty$，是为了让 softmax 后这些位置的权重精确变成 0。
为什么不在 softmax 之后再遮？因为 softmax 之后每一行已经是一组概率，直接乘 0 会破坏“总和为 1”的性质，还得额外再归一化。

notebook 先展示了一个“先 softmax 再乘下三角 mask”的直观写法，然后说明更好的做法是：**先在分数阶段用 $-\infty$ 遮掉未来位置，再进 softmax**。

代码对应

这部分对应：

- `Cell 98--110`：用三角矩阵讲清楚 causal mask；
- `Cell 112--117`：在注意力权重后加 dropout；
- `Cell 121`：实现 `CausalAttention` 类，并支持 batch 输入。

```
attn_scores = queries @ keys.transpose(1, 2)
attn_scores.masked_fill_(
    self.mask.bool()[:num_tokens, :num_tokens], -torch.inf
)
attn_weights = torch.softmax(
    attn_scores / keys.shape[-1]**0.5, dim=-1
)
attn_weights = self.dropout(attn_weights)
context_vec = attn_weights @ values
```

关于 dropout，这一章只需要记住两点：

1. 它只在训练时打开，推理时关闭；
2. 它会随机丢掉一部分注意力权重，用来减轻过拟合。

这一节真正该记住什么

causal mask 的本质不是“矩阵技巧”，而是“保证第 t 个位置只能依赖它已经见过的历史信息”。

8 问题五：为什么要从单头变成 multi-head

这一部分在解决什么

单头 attention 只有一套 Q、K、V 投影，它对“相关性”的理解只有一个视角。真实语言里，一个词同时可能关心语法关系、语义关系、指代关系、位置关系。单头不一定装得下。

先抓直觉

multi-head attention 的想法是：不要只让模型用一副眼镜看句子，而是同时戴上多副眼镜。每个 head 都有自己的一套投影参数，于是它们可以从不同子空间里抽取不同关系，最后再把结果拼起来。

数学表达

第 h 个头先单独做一次 attention：

$$\text{head}_h = \text{Attention}(Q_h, K_h, V_h).$$

然后把所有头的输出拼接起来：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)W_O.$$

这里：

- H 是 head 的数量；
- W_O 是最后把多头结果再混合一次的输出投影矩阵；
- 每个 head 通常只负责总维度中的一部分，所以会有

$$\text{head_dim} = d_{\text{out}}/H.$$

notebook 给了两个版本：

1. **概念清晰版**：`MultiHeadAttentionWrapper`，直接把多个单头 `CausalAttention` 并排跑，再把结果拼接起来。
2. **工程高效版**：`MultiHeadAttention`，先用一组大的线性层算出总的 Q、K、V，再通过 reshape 和 transpose 把最后一个维度切成多个 head 并行处理。

代码对应

这部分对应：

- `Cell 126--128`：先讲 single-head 与 multi-head 的直觉；
- `Cell 127`：wrapper 写法，容易懂；
- `Cell 131`：更标准、更高效的实现；
- `Cell 138--140`：解释四维张量上的批量矩阵乘法。

这一节还有两个容易混的点：

- wrapper 版本把多个头直接拼起来，所以输出维度会变成“每头维度 × 头数”；
- 高效版本通常把总输出维度固定成 d_{out}，再用 out_proj 做一次线性混合。

这一节真正该记住什么

multi-head 的价值不在“头越多越神奇”，而在“让模型能并行地从不同表示子空间读取不同类型的关系”。

9 把 notebook 的代码主线整理成一张表

阶段	关键 cell	你应该把它看成什么
长序列问题	8–16	为什么旧的 encoder-decoder 会遇到信息压缩瓶颈，为什么要引入 attention。
极简 self-attention	24, 28, 37, 40	一个词如何对全句打分、归一化、加权求和。
推广到整句	47, 49, 51, 55	从“算一个词的上下文”推广到“同时算所有词的上下文”。
引入 Q/K/V	66–83	从无参数玩具版过渡到真实可训练版。
封装单头模块	86, 89	把手工步骤封装成 `SelfAttention` 类。
causal mask	98–110	为什么 mask 必须在 softmax 之前作用到分数上。
dropout 与 batch	115–121	加训练期正则，并把输入扩展到 batch 维。
多头封装	127, 131	从概念清晰版过渡到高效工程版。
高维张量乘法	138–140	帮你看懂四维张量上的 `@` 到底在做什么。

10 这一章和真实 GPT 之间还差什么

学完这个 notebook，你已经拿到了 GPT 注意力模块的骨架，但距离完整 GPT 还差几块：

1. 还没有把注意力和前馈网络、残差连接、LayerNorm 组装成完整 block；
2. 还没有接上 token embedding、位置编码、输出头；
3. 还没有放进训练循环和数据加载流程里；
4. 这里的 toy 数字只是演示，不代表真实模型规模。

但从“理解 attention 模块本身”这个目标看，第 3 章已经把最关键的部分讲完了。

11 最后真正该记住什么

1. attention 的动机是解决“长序列信息被固定向量压缩丢失”的问题。
2. self-attention 的最小计算链是：打分、softmax、加权求和。
3. 真实模型用 Q、K、V，是为了让“提问、匹配、取值”都可学习。
4. causal mask 保证语言模型在当前位置不能偷看未来词。
5. multi-head 让模型能从不同表示子空间并行读取不同关系。

6. notebook 里的每段代码都只是把这条主线一步步写出来。

这一节真正该记住什么

如果你只能记一句话，那就是：**attention 就是“当前词按相关性去全序列里取信息”，而第 3 章是在把这个想法一步步变成 GPT 里真正可用的代码模块。**